

Definición última de Alcances - Inclusiones y Exclusiones

▼ Matriz de Alcance de funcionalidades, divididas en: 'Críticas para MVP', 'Deseables para Fase 2' y 'Fuera de Alcance'.

Funcionalidad	Categoría	Descripción / Límite del alcance
Gestión de Usuarios	Crítica (MVP)	Registro, login, manejo de sesiones y diferenciación estricta entre roles CLIENTE , NODO y ADMIN . Stack: Supabase Auth - gestionado desde el backend con Nest.js.
CRUD Básico de Catálogo y Nodos	Crítica (MVP)	Panel básico (solo ADMIN) para cargar productos y dar de alta Nodos físicos disponibles.
Asociación Cliente-Nodo	Crítica (MVP)	El cliente selecciona y guarda su "Nodo de preferencia".
Flujo de Grupo de Compras	Crítica (MVP)	Visualizar productos, iniciar un grupo nuevo o unirse a uno existente con cupo. Lógica para cambiar el estado del grupo a "Completado" cuando se vendan todas las unidades del bulto.
Pagos y Escrow	Crítica (MVP)	Integración con el checkout de una pasarela (como MercadoPago) al unirse al grupo. Escrow del dinero hasta completar el grupo de compra del bulto y realizar el pedido de compra + pago al mayorista. El proceso de compra real al mayorista es simulado y delegado.
Código/QR de Entregas	Crítica (MVP)	Generación de código único (alfanumérico o QR) en la app del cliente para retiro. Stack: angularx-qrcode .
Flujo del Gestor de Nodo	Crítica (MVP)	Visualizar lista de bultos entrantes, marcar como " Recibido ", escanear QR/código y marcar " Entregado ". Stack: Html5-QRCode .

Simulación de Envío	Crítica (MVP)	Botón o script (cron job) que pase grupos "Completados" a "En Nodo". Stack: <code>@nestjs/schedule</code> para los Cron Jobs.
Notificaciones Automáticas	Crítica (MVP)	Avisos por email, push o WhatsApp sobre estados del grupo y pedidos.
Billetera Virtual / Reembolsos	Crítica (MVP)	Si un grupo no se completa en X tiempo (Al final del día), devolver el dinero a una "billetera Redecos" en lugar de hacer un reembolso complejo a la tarjeta. <code>Analizar que complejidad tiene un reembolso real a la tarjeta/billetera virtual</code>
Métricas y Dashboards	Crítica (MVP)	Tres informes con gráficos por pantalla sobre ganancias, retiros y ahorro. Gráficos de ganancias para el nodo (en base a la comisión ganada por cada pedido entregado), cantidad de pedidos recibidos, tiempo promedio que tardan los clientes en retirar, historial de ahorro para el cliente, comparación del total gastado con lo que gastaría pagando el precio minorista, etc. Stack: <code>Chart.js</code> o <code>ng2-charts</code> en Angular.
Integración de APIs de Mayoristas	Fuera de Alcance	Sincronización de stock y precios descartada por limitaciones técnicas y de modelo de negocio. Se asume acuerdo de precios por 24hs.
Tracking GPS	Fuera de Alcance	Seguimiento en tiempo real del flete descartado.

Dependencias Técnicas y Arquitectónicas

1. Diseño del Modelo de Datos (La base de todo)

- **Dependencia:** Ninguna. Es el paso cero.
- **Detalle:** Modelar la relación entre `Producto` (el catálogo maestro), `Bulto` (la instancia de la compra grupal con su capacidad máxima y actual) y `Reserva_Cliente` (qué cliente compró qué porción del bulto y su estado de pago).

2. Sistema de Autenticación y Permisos

- **Depende de:** Modelo de datos de Usuarios.

- **Detalle:** El CRUD de nodos y productos, y la visualización de catálogos dependen de saber quién está haciendo la petición. Crear y asegurar los endpoints para que dependiendo del **ROL** puedan o no realizar ciertas acciones del sistema.

3. Catálogo y Gestión de Nodos (Backoffice)

- **Depende de:** Autenticación (**ADMIN**).
- **Detalle:** Sin productos cargados en la base de datos y sin al menos un nodo creado, no se puede construir ni probar la interfaz del cliente. Debo cargar un catálogo de productos en la base de datos de prueba (para mostrar todos los productos del catálogo) y crear al menos dos nodos (para mostrar que bultos están en proceso de compra por grupos de clientes de un nodo exclusivamente, a que nodo asociar la compra de un cliente, etc.)

4. Lógica Transaccional del "Grupo de Compra"

- **Depende de:** Catálogo y Autenticación de Cliente.
- **Detalle:** Esta es la funcionalidad core. La dependencia técnica más crítica aquí es el manejo de concurrencia. ¿Qué pasa si dos clientes intentan comprar la última unidad del bulto exactamente al mismo tiempo? El backend debe manejar transacciones de base de datos seguras (bloqueos optimistas/pesimistas) para evitar sobreventas. También tener en cuenta las relaciones del cliente con su nodo, visualizar los pedidos creados desde el nodo, etc.

5. Integración del Pago

- **Depende de:** Lógica del Grupo de Compra.
- **Detalle:** El pago solo debe procesarse si hay "cupó" en el bulto. El cambio de estado del cliente a "Pagado" y el aumento del contador del bulto deben ocurrir únicamente tras la confirmación exitosa de la pasarela de pagos (escuchando un webhook, por ejemplo → **INVESTIGAR**).

6. Generación y Validación de QR/Códigos

- **Depende de:** Un grupo de compra con estado "Completado" y "Pedido en el Nodo XX".
- **Detalle:** A nivel UI, necesitarás una librería específica para renderizar el QR en el frontend del cliente. El contenido del QR debe ser un token seguro o un ID encriptado, no simplemente el ID de la base de datos,

para evitar que alguien adivine códigos ajenos. O implementar código de confirmación como RAPPI/PEDIDOS YA. (Un código que tiene el cliente, el cual el repartidor debe cargar para confirmar la entrega del pedido).

7. Flujo de Recepción del Nodo

- **Depende de:** Generación de QR y simulación de envío.
- **Detalle:** El frontend del nodo necesita un escáner (usando la cámara del dispositivo) que lea el QR generado en el paso anterior (O ingresar manualmente el código del pedido) y envíe una petición al backend para hacer el cruce de datos y cerrar el ciclo, marcando los productos comprados por cada cliente como entregados y terminando el proceso.

▼ Roadmap de ejecución - MVP Redecos

Sprint 0: Arquitectura Base y Entornos

Finaliza: Miércoles 20/05/2026

- **Épica:** *Foundation & Setup*
- **Tareas Principales:**
 - Redefinición de alcances para el MVP.
 - Diseño del modelo de datos relacional en Supabase.
 - Inicialización de repositorios en GitHub.
 - Configuración del backend en Nest.js y scaffolding del frontend en Angular + Ionic.
- **Hito de Entrega:**
 - Entornos de desarrollo levantados y conectados.

Sprint 1: Identidad, Roles y Backoffice

Jueves 21/05 al Miércoles 03/06

- **Épica:** *Core System & Admin Privileges*
- **Tareas Principales:**
 - Implementación de registro y login con Supabase Auth.
 - Restricción de rutas por roles (CLIENTE, NODO, ADMIN) mediante Guards en Nest.js.

- Panel de **ADMIN** para CRUD del catálogo y Nodos.
- **Hito de Entrega:** Autenticación funcional. Base de datos poblada con al menos dos nodos habilitados y productos semilla para pruebas.

Sprint 2: Ecosistema del Cliente

Jueves 04/06 al Miércoles 17/06

- **Épica:** *Client Onboarding & Discovery*
- **Tareas Principales:**
 - Flujo para vinculación del usuario con su nodo de preferencia.
 - Interfaz UI para exploración del catálogo y grupos de compra activos.
 - Integración de los primeros gráficos de "Métricas y Dashboards" del lado del cliente.
- **Hito de Entrega:** El cliente navega fluidamente, selecciona su nodo y visualiza los bultos en proceso de compra.

Sprint 3: Transacciones y Concurrency

Jueves 18/06 al Miércoles 01/07

- **Épica:** *The Buying Engine*
- **Tareas Principales:**
 - Lógica para unirse o crear grupos de compra.
 - Prevención de sobreventas con bloqueos en la base de datos.
 - Integración del mock de pasarela de pagos con *escrow* lógico en Nest.js.
 - Cambio de estado automático a "**Grupo Completado**".
- **Hito de Entrega:** Motor operativo donde el cliente simula la compra y el sistema bloquea los cupos agotados.

Sprint 4: Logística, Billetera y Notificaciones (Lado Cliente)

Jueves 02/07 al Miércoles 15/07

- **Épica:** *Fulfillment Prep & Verification*

- **Tareas Principales:**

- Simulador/Cron Job para enviar bultos a "En Nodo".
- Generación de código único/QR en la app cliente.
- Desarrollo del CRON de "Billetera Redecos" para reembolsar dinero si un grupo fracasa al final del día.
- Implementación de las notificaciones automáticas (email/push)

- **Hito de Entrega:** Transición logística resuelta. Cliente con credencial digital y motor de reembolsos/notificaciones en segundo plano activado.

Sprint 5: Recepción, Cierre y Pulido

Jueves 16/07 al Miércoles 29/07

- **Épica:** *Node Operations & Final Polish*

- **Tareas Principales:**

- Interfaz del Gestor de Nodo para confirmar recepción.
- Lógica de escáner de cámara Ionic/QR cruzada con el backend.
- Implementación de los Dashboards y métricas del Nodo.
- Ajustes responsivos con Tailwind CSS.

- **Hito de Entrega:** MVP consolidado y listo para presentación. Flujo 100% funcional validado desde el registro de usuario hasta la entrega física de la mercadería, listo para su presentación.

▼ Épicas

SPRINT 1: Configuración Core y Control de Acceso

Meta del Sprint: "Levantar la arquitectura base del proyecto, conectar la base de datos y asegurar el sistema de autenticación con diferenciación de roles para tener el primer entorno funcional."

1. Épicas asociadas al Sprint

- Gestión de Usuarios e Identidad (Autenticación y Roles).

2. Historias de Usuario (HU)

- **HU-1.1:** Como **Administrador**, quiero **iniciar sesión en un panel seguro para gestionar el sistema sin que usuarios no autorizados tengan acceso**.
 - *Criterios de Aceptación:*
 1. El login debe validar credenciales contra Supabase Auth.
 2. Si un usuario sin rol ADMIN intenta ingresar a las rutas de administración, el sistema debe redirigirlo y mostrar un error `401 Unauthorized`.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Proyecto en Supabase creado. Módulo de Autenticación en NestJS configurado con JWT Guard y un `RolesGuard` personalizado.
 - **Frontend:** Proyecto base Ionic/Angular generado. Pantalla de Login maquetada y ruteo protegido con Angular Route Guards.
- **HU-1.2:** Como **Cliente**, quiero **registrarme en la aplicación creando un usuario para poder utilizar los servicios de Redecos**.
 - *Criterios de Aceptación:*
 1. El formulario debe pedir datos básicos (Email, Contraseña, Nombre).
 2. Al registrarse, la base de datos debe asignar automáticamente el rol `CLIENTE`.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Endpoint `POST /auth/register` en NestJS. Tabla `users` mapeada en PostgreSQL (Supabase) con RLS (Row Level Security) básica.
 - **Frontend:** Formulario de registro conectado al servicio HTTP.

SPRINT 2: Gestión de Catálogo y Ecosistema de Nodos

Meta del Sprint: "Proveer al administrador las herramientas para cargar la oferta (productos y nodos) y permitir al cliente descubrir el catálogo y asociarse a su punto de retiro."

1. Épicas asociadas al Sprint

- CRUD Básico de Catálogo y Nodos.
- Asociación Cliente-Nodo.

2. Historias de Usuario (HU)

- **HU-2.1:** Como **Administrador**, quiero **dar de alta, editar y eliminar productos y nodos de distribución** para **mantener la oferta actualizada**.
 - *Criterios de Aceptación:*
 1. Se deben poder cargar productos definiendo: nombre, precio y tamaño del bulto.
 2. Se deben poder dar de alta Nodos físicos.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Tablas `products` y `nodes` .
Controladores REST en NestJS (`GET`, `POST`, `PUT`, `DELETE`) protegidos solo para ADMIN.
 - **Frontend:** Vistas de tabla/lista en el panel backoffice para gestionar estas dos entidades.
- **HU-2.2:** Como **Cliente**, quiero **seleccionar y guardar mi nodo de preferencia** para **ver los grupos de compra y productos que llegarán cerca de mi domicilio**.
 - *Criterios de Aceptación:*
 1. El cliente debe ver un listado o mapa de nodos disponibles.
 2. Al seleccionar uno, este debe quedar guardado en su perfil de usuario.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Relación foránea en la tabla `users` apuntando a `nodes` .
 - **Frontend:** Pantalla de "Elegir Nodo" con selector. Uso de *Signals* para mantener el nodo seleccionado en el estado global de la PWA.
- **HU-2.3:** Como **Cliente**, quiero **explorar el catálogo de productos disponibles en mi nodo** para **decidir qué compras realizar**.

- *Criterios de Aceptación:*
 1. Mostrar la grilla de productos.
 2. Indicar claramente el precio y la cantidad de unidades que componen el bulto cerrado.
- *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Endpoint `GET /catalog?nodeId={id}`.
 - **Frontend:** Vista principal (Home) del cliente renderizando la lista de productos de forma responsive.

SPRINT 3: Motor Transaccional y Grupo de Compras (CORE)

Meta del Sprint: "El corazón del negocio: Permitir la creación de grupos de compra, manejar la concurrencia para evitar sobreventas y retener el pago mediante Mercado Pago."

1. Épicas asociadas al Sprint

- Flujo de Grupo de Compras.
- Pagos y Escrow.

2. Historias de Usuario (HU)

- **HU-3.1:** Como **Cliente**, quiero **iniciar un grupo de compra nuevo o unirme a uno existente** para **adquirir mi parte de un bulto mayorista**.
 - *Criterios de Aceptación:*
 1. El sistema debe restar correctamente el cupo disponible del bulto.
 2. Si dos usuarios intentan comprar el último cupo al mismo tiempo, solo uno debe tener éxito (prevención de concurrencia).
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Tabla transaccional `purchase_groups` y `group_members`. Bloqueos de fila (Row-level locks / *Optimistic Locking*) en Supabase a través de NestJS.
 - **Frontend:** Pantalla de detalle del producto mostrando una barra de progreso de "Cupos Disponibles".

- **HU-3.2:** Como **Sistema**, quiero **procesar el pago con Mercado Pago en modo Escrow (Autorización y Captura)** para **garantizar los fondos sin cobrarlos hasta que el grupo se complete**.
 - *Criterios de Aceptación:*
 1. Al unirse, el cliente debe atravesar un checkout que congele el saldo en su tarjeta.
 2. El grupo de compra debe cambiar su estado a "Completado" automáticamente cuando se venda la última unidad.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Integración SDK Mercado Pago en NestJS (creación de Intención de Pago con `capture: false`). Webhook listener para recibir aprobaciones.
 - **Frontend:** Redirección al checkout de Mercado Pago (o integración de Brick) y pantalla de "Pago Exitoso".

SPRINT 4: Logística, Códigos Digitales y Billetera (Background Jobs)

Meta del Sprint: "Automatizar el flujo logístico, la generación de credenciales digitales para retiros y el sistema de reembolsos si un grupo fracasa."

1. Épicas asociadas al Sprint

- Simulación de Envío.
- Billetera Virtual / Reembolsos.
- Código/QR de Entregas.

2. Historias de Usuario (HU)

- **HU-4.1:** Como **Sistema**, quiero **ejecutar un proceso automático diario** para **pasar los grupos completados a "En Nodo" o reembolsar los incompletos**.
 - *Criterios de Aceptación:*
 1. Los grupos completados mutan de estado simulando el envío logístico.

2. Los grupos que no se llenaron al final del día se cancelan y se dispara la orden de descongelar los fondos en Mercado Pago (Reembolso).
- *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Uso de `@nestjs/schedule` para Cron Jobs nocturnos que interactúen con la API de Mercado Pago.
 - **Frontend:** Ninguno directo (proceso de backend), pero los estados se reflejarán en el historial del cliente.
 - **HU-4.2:** Como **Cliente**, quiero **obtener un código QR una vez que mi pedido llega al nodo para retirar mis productos de forma segura**.
 - *Criterios de Aceptación:*
 1. El QR solo debe generarse/mostrarse si el estado del pedido es "En Nodo".
 2. El código embebido debe ser seguro y único por cliente/reserva.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Generación de un UUID/Token único en la reserva.
 - **Frontend:** Implementación de la librería `angularx-qrcode` en la vista de "Mis Pedidos".
 - **HU-4.3:** Como **Cliente y Nodo**, quiero **visualizar métricas de mis operaciones para conocer mi ahorro y mis ganancias respectivamente**.
 - *Criterios de Aceptación:*
 1. El cliente debe ver su historial de ahorro.
 2. El Nodo debe ver al menos un gráfico de ganancias por comisión.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Endpoints de analíticas en NestJS utilizando consultas SQL de agregación.
 - **Frontend:** Implementación de `Chart.js` o `ng2-charts` en los paneles correspondientes.

SPRINT 5: Operaciones del Nodo, Pulido Final y Despliegue

Meta del Sprint: "Cerrar el ciclo físico permitiendo al Nodo escanear entregas, pulir la experiencia de usuario general (notificaciones) y desplegar la PWA para la defensa."

1. Épicas asociadas al Sprint

- Flujo del Gestor de Nodo.
- Notificaciones Automáticas.
- Cierre, Testing E2E y Despliegue.

2. Historias de Usuario (HU)

- **HU-5.1:** Como **Gestor de Nodo**, quiero **escanear el QR del cliente** para **marcar sus productos como "Entregados" y finalizar el ciclo.**
 - *Criterios de Aceptación:*
 1. La PWA debe abrir la cámara del dispositivo móvil del Gestor de Nodo de forma nativa en la web.
 2. Al leer el código correcto, el backend debe registrar la entrega y actualizar el dashboard de ganancias del nodo.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Endpoint `POST /deliveries/scan` que reciba el token y cruce datos.
 - **Frontend:** Integración de la librería `html5-qrcode` para acceso a la cámara vía WebRTC en Angular.
- **HU-5.2:** Como **Usuario (Cliente/Nodo)**, quiero **recibir notificaciones automáticas** para **estar enterado de los cambios de estado de mi grupo y mi pedido.**
 - *Criterios de Aceptación:*
 1. Enviar alerta cuando el grupo se completa.
 2. Enviar alerta cuando el pedido está listo para retirar en el nodo.
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** Integración de envío de Emails transaccionales o Firebase Cloud Messaging.

- **Frontend:** Si se usa FCM, configuración de Service Workers en la PWA para recibir *Push Notifications*.
- **HU-5.3:** Como **Desarrollador**, quiero **realizar pruebas, pulir la UI/UX y desplegar la aplicación a producción** para **garantizar una demostración final sin fallas**.
 - *Criterios de Aceptación:*
 1. El flujo crítico de compra debe funcionar sin bugs en un entorno cloud, no en `localhost`.
 2. La UI debe verse bien en dispositivos móviles (responsividad).
 - *Entregables Técnicos Vinculados:*
 - **Base de Datos / Backend:** NestJS desplegado en Render/Railway. Base de datos productiva en Supabase.
 - **Frontend:** PWA compilada y desplegada en Vercel o Firebase Hosting con certificado HTTPS.
- Flujo de Grupo de Compras.
- Pagos y Escrow.
- Simulación de Envío.
- Billetera Virtual / Reembolsos.
- Código/QR de Entregas.
- Flujo del Gestor de Nodo.
- Notificaciones Automáticas.
- Cierre, Testing E2E y Despliegue.